Nitrostack Hackathon 2026 | Healthtech & Open Innovation

# SynapseMed

## HEALTHCARE CONTEXT INTELLIGENCE ENGINE (HCIE) V0.3.1

Optimized Agentic Orchestration for Medical Emergencies. Connecting unstructured requests to secure, open-standard knowledge structures via the Model Context Protocol.

---

| LANGUAGE / RUNTIME | ARCHITECTURE STATUS | PROJECT SANDBOX |
| --- | --- | --- |
| **Python 3.11+ / MCP-Native** | **Optimized Blueprint v0.3.1** | **Open-Source Core** |

# 01 / Vision & First Principles

The conceptual shift from basic resource retrieval to clinical-grade medical context synthesis.

# First Principle: Context, Not Diagnosis

### The Challenge

Healthcare systems struggle with extremely noisy, non-standardized emergency requests and highly fragmented, legacy EHR systems.

AI models face severe hallucination risks when acting as clinical classifiers or making unsupported treatment suggestions.

### The HCIE Solution

**Context synthesis, not medical diagnosis.** HCIE operates between unstructured requests and structured medical networks to build optimal context graphs.

It provides responders with secure execution plans and clear rationale—while keeping medicine strictly with clinicians.

# Architectural Evolution (v0.3.1)

| Architectural Dimension | v0.3 Legacy Pipeline | v0.3.1 Optimized Pipeline | Why It Matters (Impact) |
| --- | --- | --- | --- |
| **Agent Hierarchy** | 9 Wrapper Agents (high latency hops) | **2 Core Agents (Chief & Execution)** | Collapses redundant API hops; isolates execution authority. |
| **Discovery Flow** | Serial/Sequential Chain calls | **Async/Concurrent Parallel dispatch** | Discovery latency is bound to max() instead of sum(). |
| **File Placement** | Orphaned top-level folders | Deep integration (Context & Retrieval) | Strict one-concept-per-folder architecture. |
| **Est. File Footprint** | ~85-95 source files | **~78-85 source files** | Easier to validate, compile, and maintain on servers. |

# Input Transformation Workflow

## Extracting Intent, Guiding Action

- **Unstructured Query:** Raw symptom logs, coordinates, or emergency dispatch calls.
- **Machine-Operable Intent:** Translates symptoms to emergency categories with rules-based severity hints.
- **Safe Execution Plan:** Proposes action points (e.g. notify family, route ambulance, retrieve EHR records).
- **No Diagnostic Spills:** Never outputs medication dosages or disease conclusions.

LIVE BLUEPRINT PAYLOAD


```
{ "task": "emergency_orchestration", "emergency_type":
"cardiac_suspected", "patient_name": "John Doe",
"symptoms": ["chest_pain", "unconscious"], "location":
"Kakkanad", "severity_hint": "high" }
```

# 02 / Core Architecture & MCP

How the Model Context Protocol (MCP) bridges the gap between LLMs and sensitive medical records.

# Three Coordination Zones

### 1. Client Layer

Consists of standard clinician interfaces, Claude Desktop, or custom responder frontends that speak standard JSON-RPC/SSE.

### 2. Secure Gateway

Auth validation, capability discovery, tool route parsing, logging, and execution locks. Contains the human-in-the-loop approval gate.

### 3. Context Engine

Calculates the Context Intelligence Score, shapes the reasoning graph, and outputs structured, validated, safe clinical resource targets.

# The 7-Layer Orchestration Engine

- **Layer 1: Intent Understanding** Parses raw text into strict classifications and severity levels.
- **Layer 2: Async Planning** Generates the query execution graph.
- **Layer 3: Knowledge Discovery** Dispatches concurrent requests to independent MCP nodes.
- **Layer 4: Hybrid Retrieval** Blends semantic embeddings and BM25 scores.
- **Layer 5: Context Graph** Assembles connected entities (Patient → Doctor → Hospital) inside memory.
- **Layer 6: Prompt Generation** Generates context-dense LLM instructions.
- **Layer 7: Guardrail Validation** Runs strict clinical validation; blocks non-compliant outputs.

# Discovery Latency Optimization

## Execution Flow Metrics: Sequential vs. Parallel (Concurrent)

Under the v0.3.1 design, four independent lookup threads run asynchronously, bounding overall latency to the slowest individual service.


Optimized with `asyncio.gather`, slashing API transaction bottlenecks by up to 72%.

# Algorithmic Infrastructure

## Context Intelligence Score

Dynamically ranks candidates by synthesizing text similarity, department availability, geographical proximity, and operational status.

**MathML Implementation Core:**

$$CIS = 0.35S_{sim} + 0.25P_{prox} + 0.20D_{dept} + 0.10R_{res}$$

## Priority & Inference Logic

Translates patient history & current symptoms into routing rules before prompt construction (e.g., matching prior cardiac issues directly to ICU units).

```
def infer_priority_resources(context_graph): priorities = []
if has_symptom("chest_pain") and has_history("heart_disease"):
priorities.append("cardiology_priority") if
has_symptom("unconscious"): priorities.append("icu_priority")
return priorities
```

# Future Ecosystem Expansion

### Biometric Edge

Smart watch compatibility for real-time telemetry (acute heart rate spikes, fall detection) to automatically trigger emergency context assembly.

### Logistics Hub

Extending open MCP server endpoints to medicine delivery systems, automated drone dispatches, and responsive ambulance marketplaces.

### Clinical EHR Integration

Replacing mock JSON files with secure, production-grade HL7/FHIR gateways mapping actual electronic medical records behind stable tool schemas.

# The Engineering Roadmap

### Hackathon MVP

Python engine core, 4 mock MCP servers, intent parser, and basic sequential orchestration.

### v0.3.1 Core

Async parallel discovery, hybrid BM25 search, context graph assembly, and validation gates.

### Clinical Pilot

Session-aware multi-turn cases, real hospital capacity metrics, and physician feedback loops.

### EHR Scale

FHIR-compliant connectors, enterprise authorization (OAuth2), and HIPAA-hardened guardrails.

Secure Open Innovation for Healthtech

# Orchestrating Healthcare Context

A standards-based, MCP-native approach to connecting LLMs safely to emergency records without medical diagnostic leakage.

GITHUB REPOSITORY
github.com/open-hcie/core

CONTACT / CORE TEAM
build@hcie.dirmacs.org

*Disclaimer: All database items, physicians, and hospitals used in current v0.3.1 are mock-simulated data. HCIE is an orchestration proxy framework and does not provide active medical diagnoses or clinical triage.*